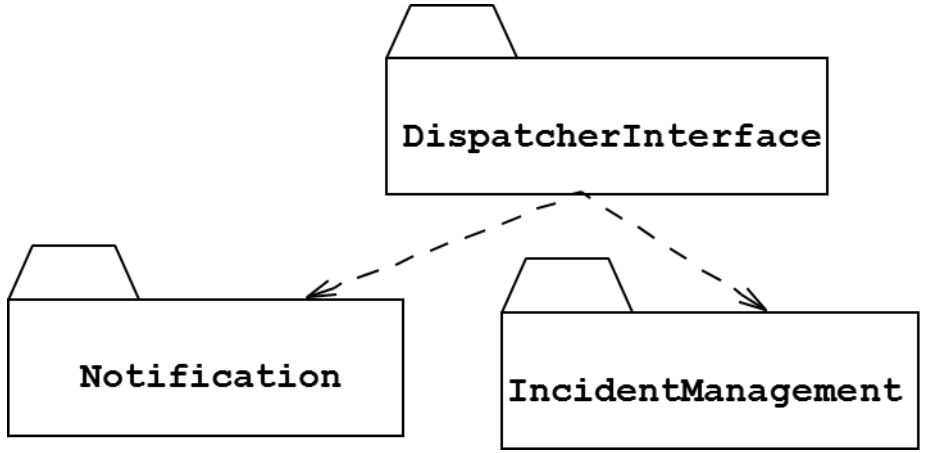
CSE 4513

Lec – 7 (2)

Modeling with UML

PACKAGE DIAGRAM

- To organize complex class diagrams, you can **group classes** into packages.
- A package is a collection of logically related UML elements
- Notation
  - Packages appear as rectangles with small tabs at the top.
  - The package name is on the tab or inside the rectangle.
  - The dotted arrows are dependencies. One package depends on another if changes in the other could possibly force changes in the first.
  - Packages are the basic grouping construct with which you may organize UML models to increase their readability



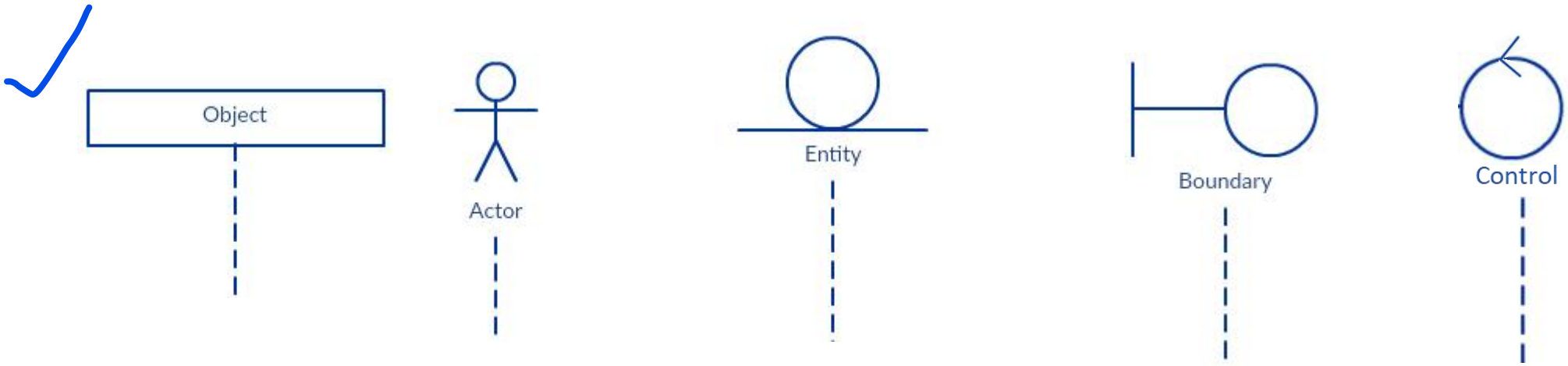
SEQUENCE DIAGRAMS

- ✓ A **Dynamic Diagram** to record Messages and their sequences
- ✓ It models the interactions between objects in a **single use case**..
- ✓ They illustrate how the different parts of a system interact with each other to carry out a function, and the **order in which the interactions occur** when a particular use case is executed.
- ✓ shows how different parts of a system work in a 'sequence' to get something done.

SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: *Lifeline Notation*

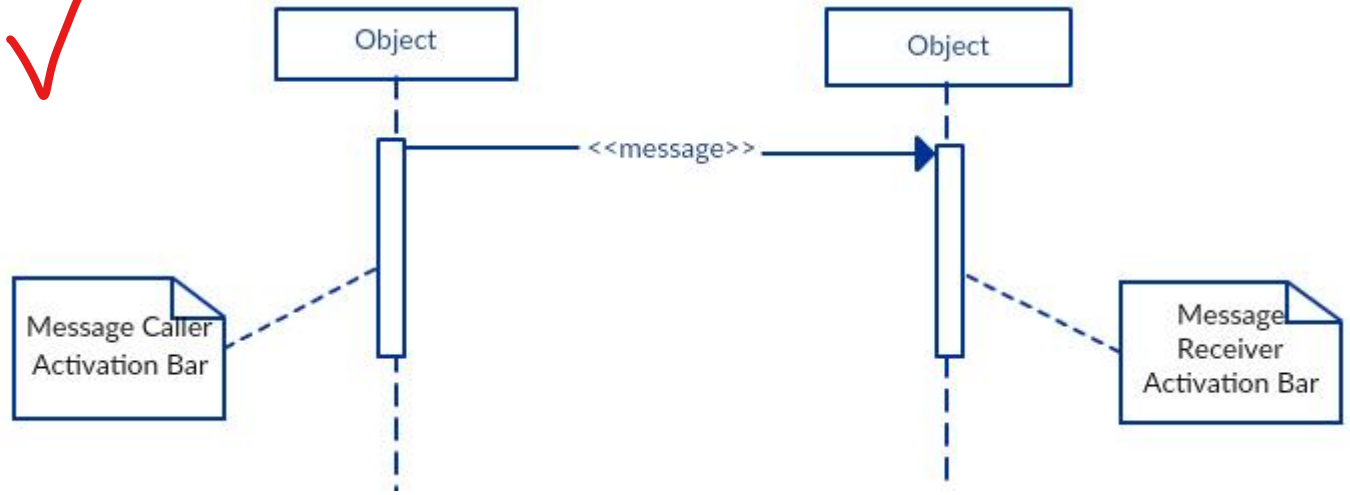
- ✓ A sequence diagram is made up of several of these lifeline notations that should be arranged horizontally
- ✓ A lifeline extends vertically downward from the participant's box, represented as a dashed line.
- ✓ The vertical line represents the participant's existence over time during the interaction.
- ✓ No two lifeline notations should overlap each other.
- ✓ They represent the different objects or parts that interact with each other in the system during the sequence.



SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: Activation Bars

- ✓ Thin vertical rectangles, or activation bars, appear on the lifeline to show when the participant is actively executing a process.
- ✓ These bars are placed where the participant is performing an operation or handling an event.
- ✓ The length of the rectangle indicates the duration of the objects staying active.
- ✓ An interaction between two objects occurs when one object sends a message to another.
- ✓ The use of the activation bar on the lifelines of the Message Caller (the object that sends the message) and the Message Receiver (the object that receives the message) indicates that both are active/ are instantiated during the exchange of the message.

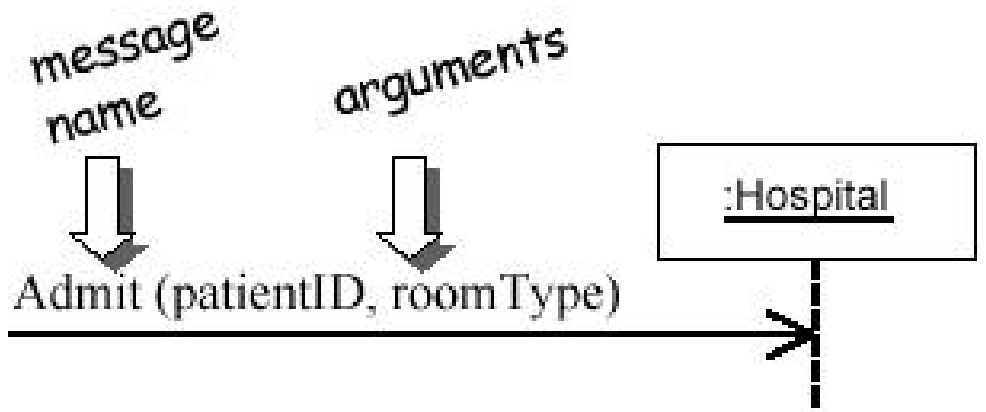


SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: *Message Arrows*

- ✓ An arrow from the Message Caller to the Message Receiver specifies a message in a sequence diagram
- ✓ A message can flow in any direction; from left to right, right to left, or back to the Message Caller itself.
- ✓ different arrowheads are used to indicate the type of message being sent or received.
- ✓ The message arrow comes with a description, which is known as a **message signature**, on it.
- ✓ The format for this message signature is below. All parts except the message_name are optional.

attribute = message_name (arguments): return_type



SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: *Message Arrows*

Synchronous Message:

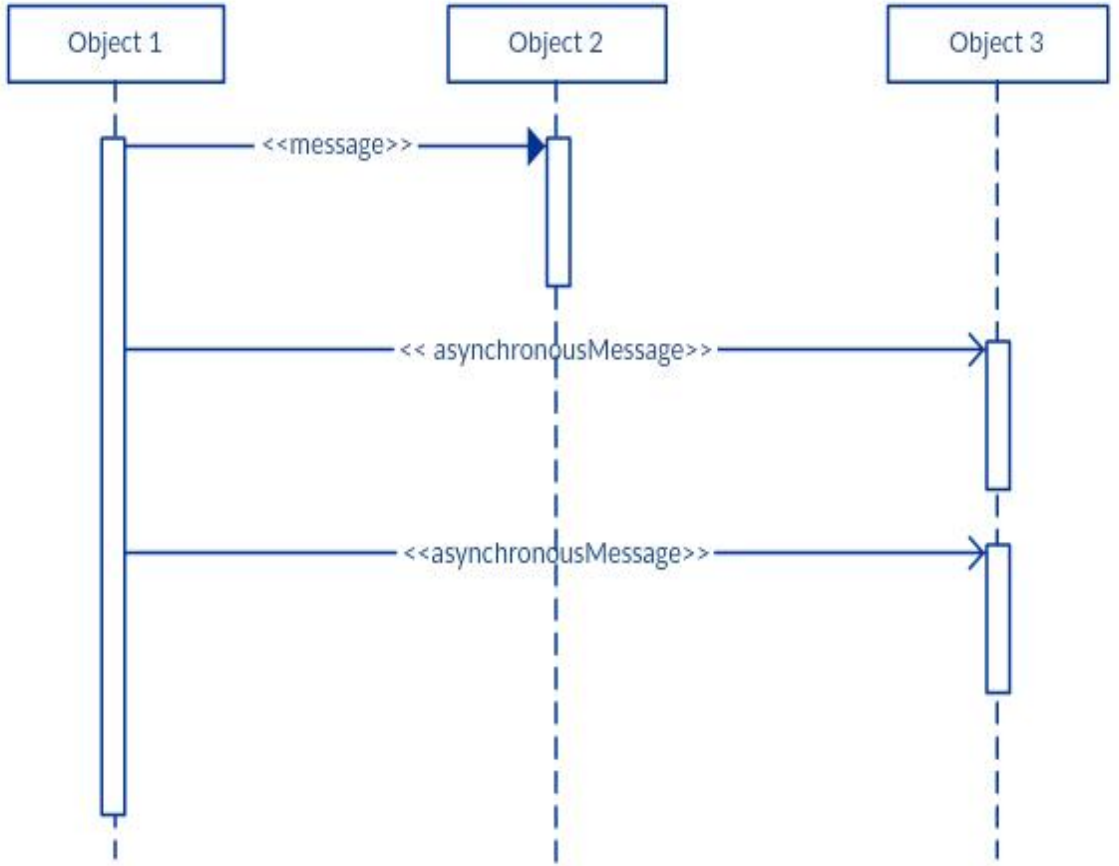
- ✓ a synchronous message is used when the sender waits for the receiver to process the message and return before carrying on with another message.
- ✓ The arrowhead used to indicate this type of message is a **solid** one, like the one below.

SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: *Message Arrows*

Asynchronous Message:

- ✓ An asynchronous message is used when the message caller does not wait for the receiver to process the message and return before sending other messages to other objects within the system.
- ✓ are represented with **solid lines** but **open arrowheads**.

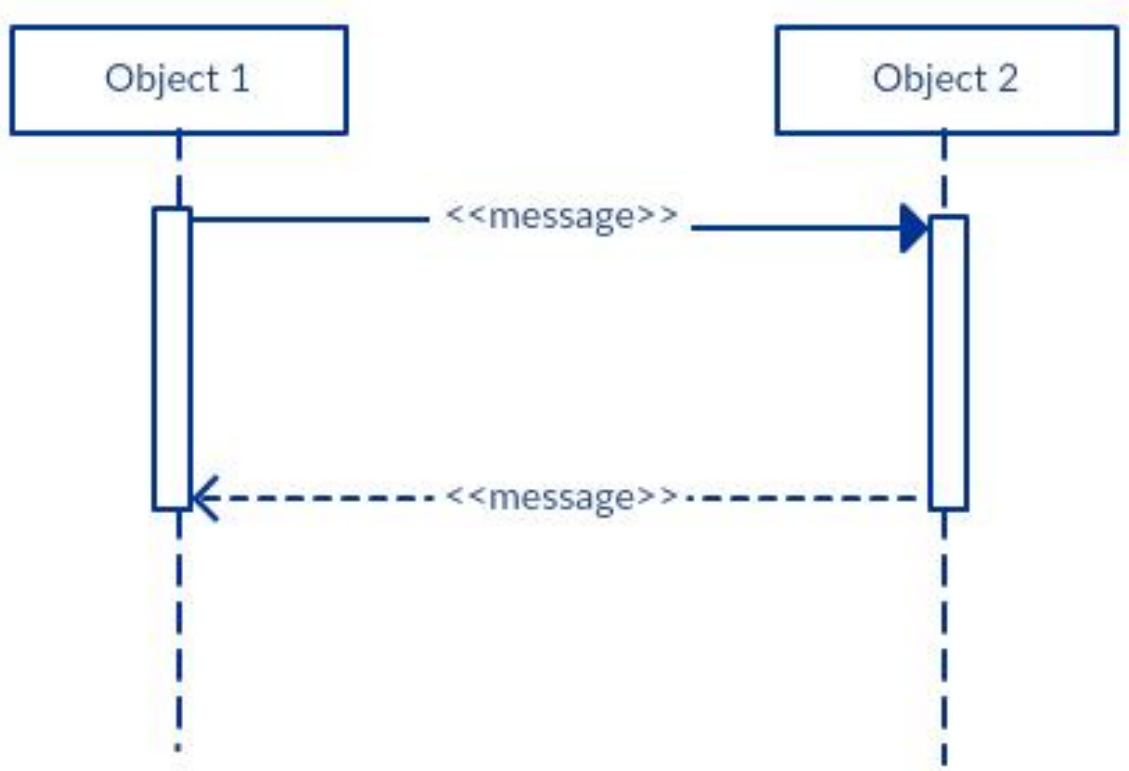


SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: *Message Arrows*

Return Message:

- ✓ A return message is used to indicate that the message receiver is done processing the message and is returning control over to the message caller.
- ✓ A return message is illustrated using a dashed line with an open arrowhead

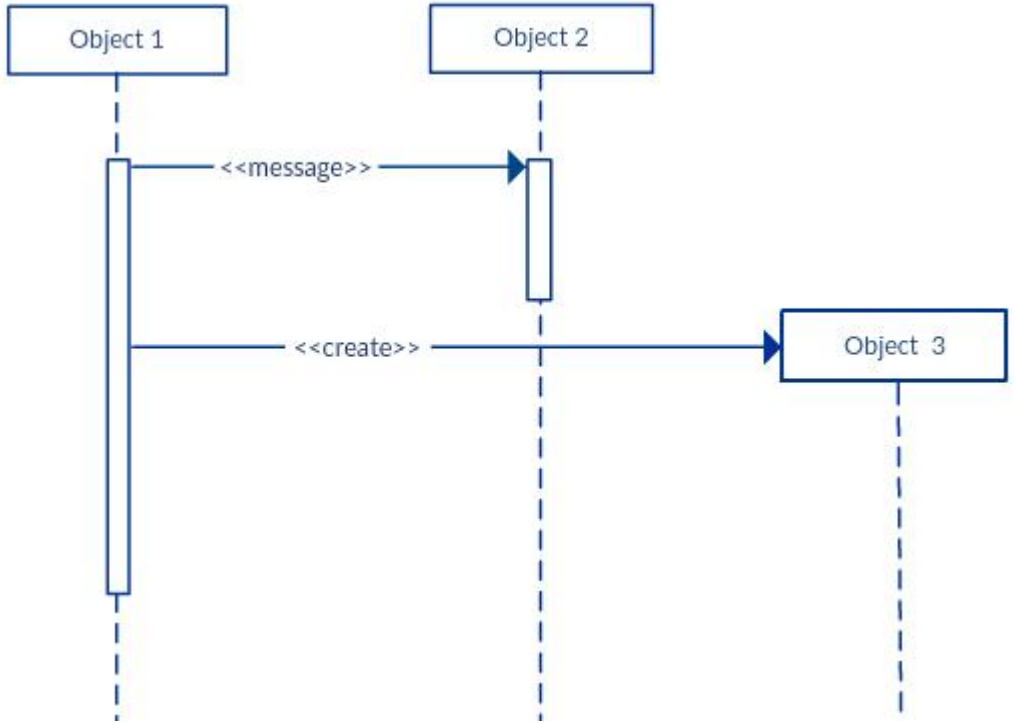


SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: *Message Arrows*

Participant creation Message:

- ✓ Objects do not necessarily live for the entire duration of the sequence of events.
- ✓ Objects or participants can be created according to the message that is being sent..

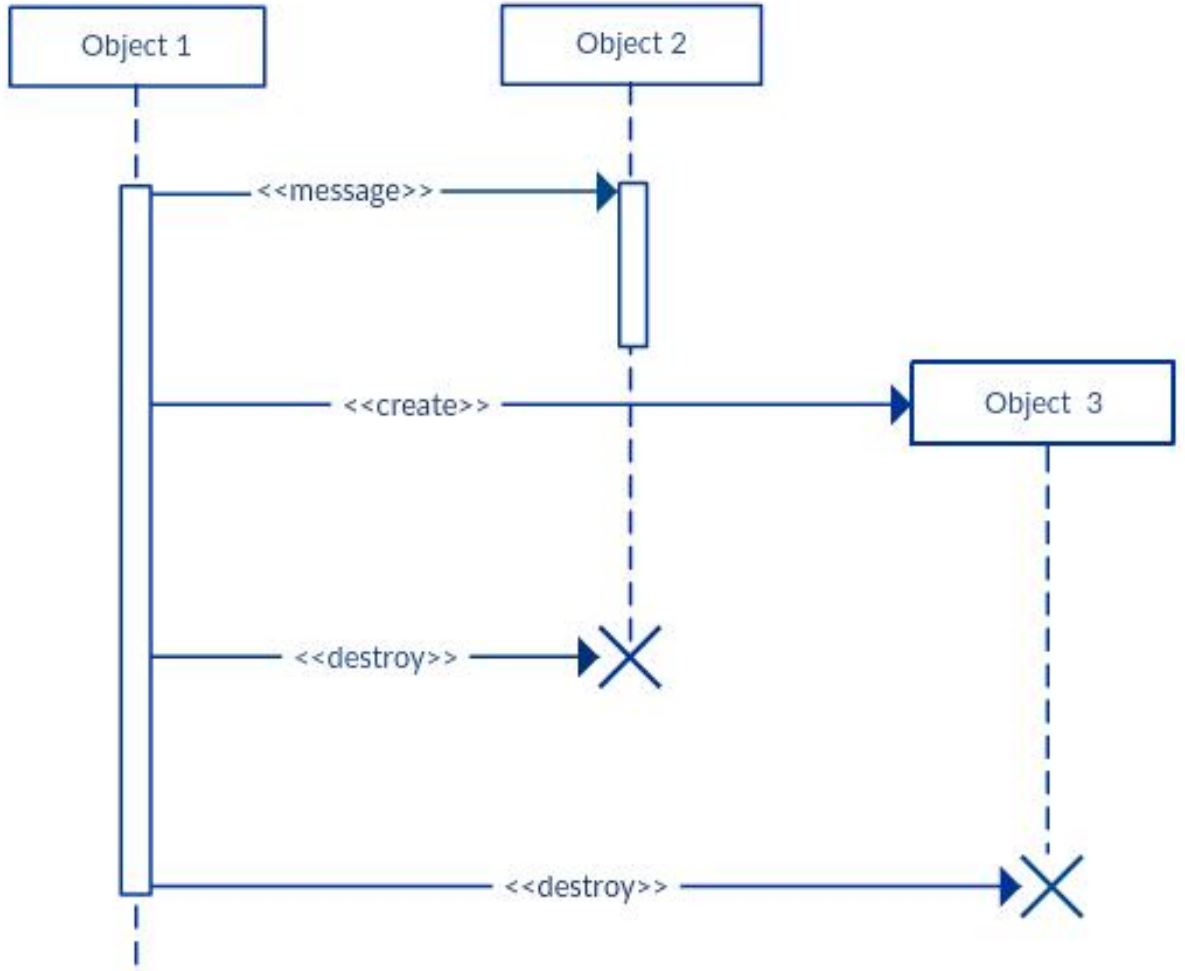


SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: *Message Arrows*

Participant destruction message :

- ✓ Likewise, participants when no longer needed can also be deleted from a sequence diagram.
- ✓ This is done by adding an 'X' at the end of the lifeline of the said participant.

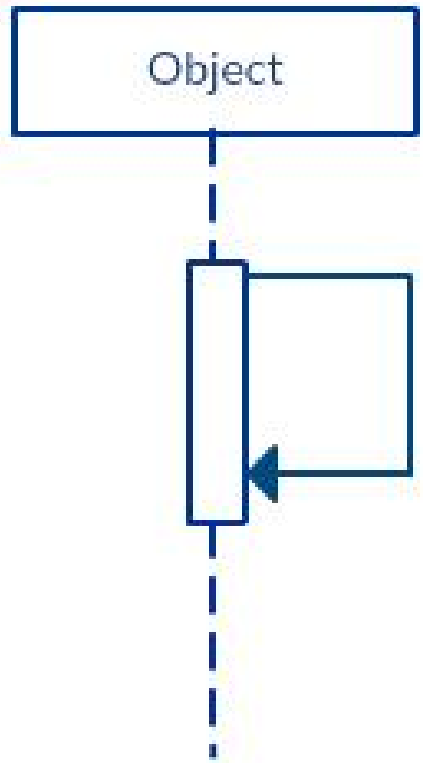


SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: *Message Arrows*

Reflexive message :

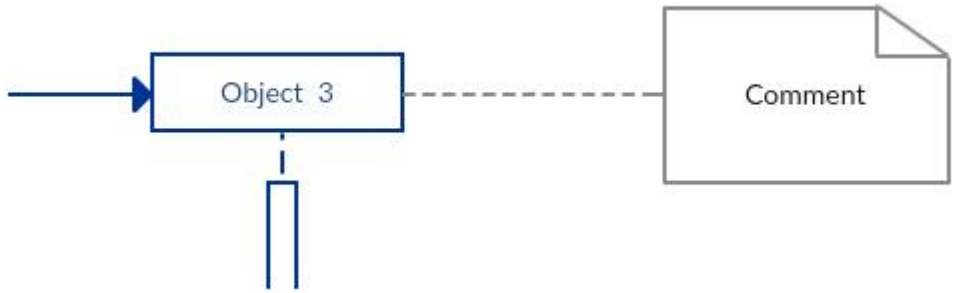
- ✓ When an object sends a message to itself, it is called a reflexive message.
- ✓ It is indicated with a message arrow that starts and ends at the same lifeline as shown in the example.



SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: *Comment*

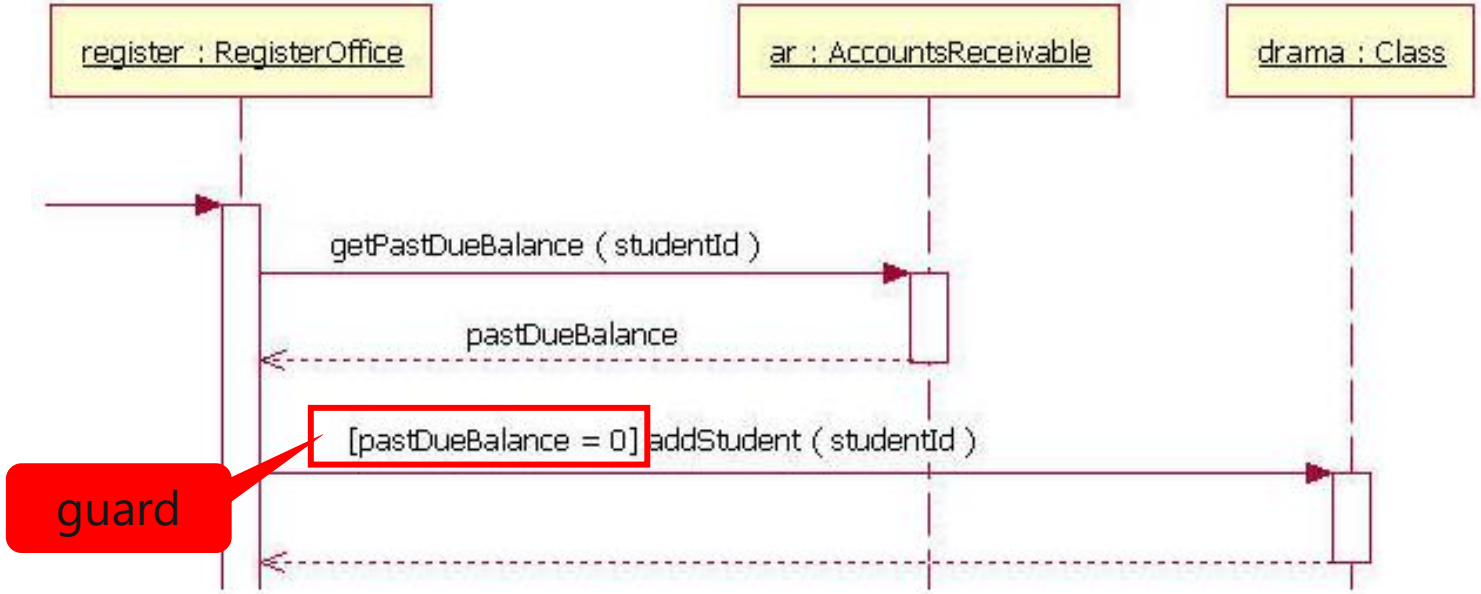
- ✓ UML diagrams generally permit the annotation of comments in all UML diagram types.
- ✓ The comment object is a rectangle with a folded-over corner as shown below.
- ✓ The comment can be linked to the related object with a dashed line.



SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: *Combined Fragment*

- ✓ Combined fragment is an interaction fragment which defines a combination (expression) of interaction fragment.
- ✓ Combined fragment may have **interaction constraints** also called **guards**.
- ✓ Guards are used throughout UML diagrams to control flow.
- ✓ To draw a **guard** on a sequence diagram in UML, you placed the guard element above the message line being guarded and in front of the message name.

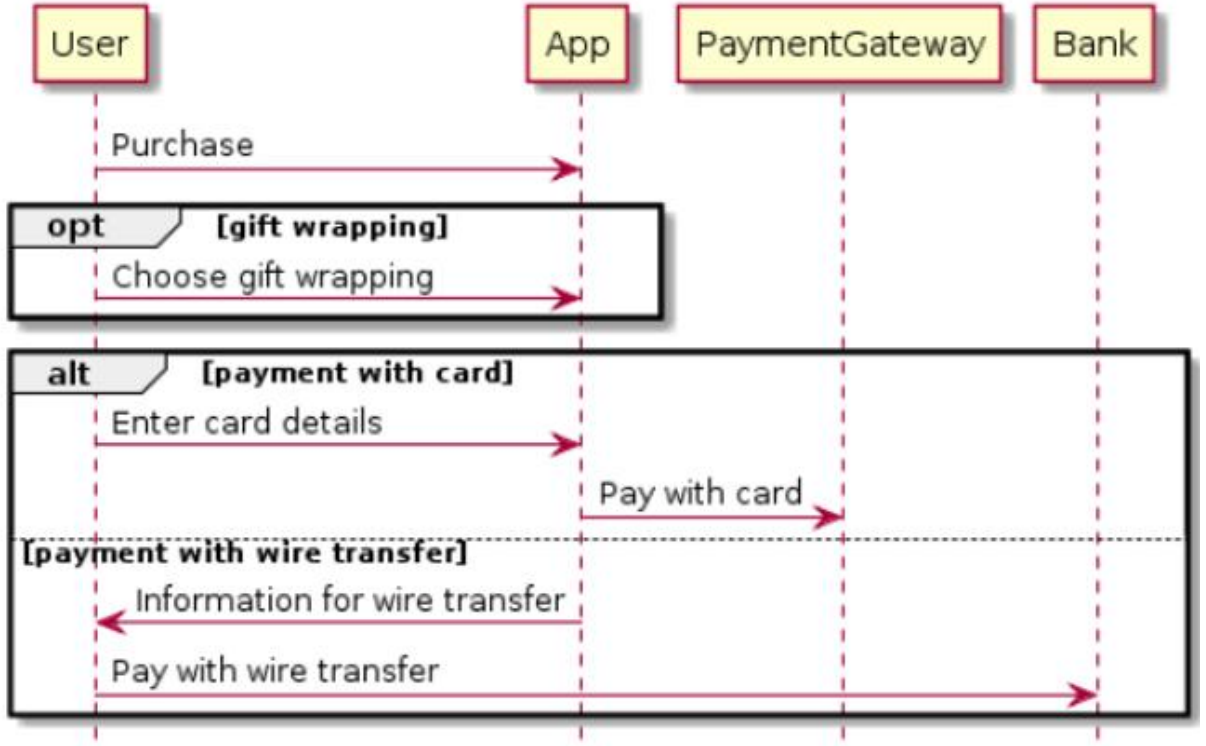


SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: *Combined Fragment*

- ✓ Frequently used Combined fragments are:
  - alt – alternatives (if else)
  - opt – option (single condition)
  - loop - iteration
  - ref – reference

- ✓ alt means that the combined fragment represents a choice or alternatives of behavior.
- ✓ Alternative fragment models if…then…else constructs
- ✓ The alternative fragment is represented by a large rectangle or a frame; it is specified by mentioning 'alt' inside the frame's name box

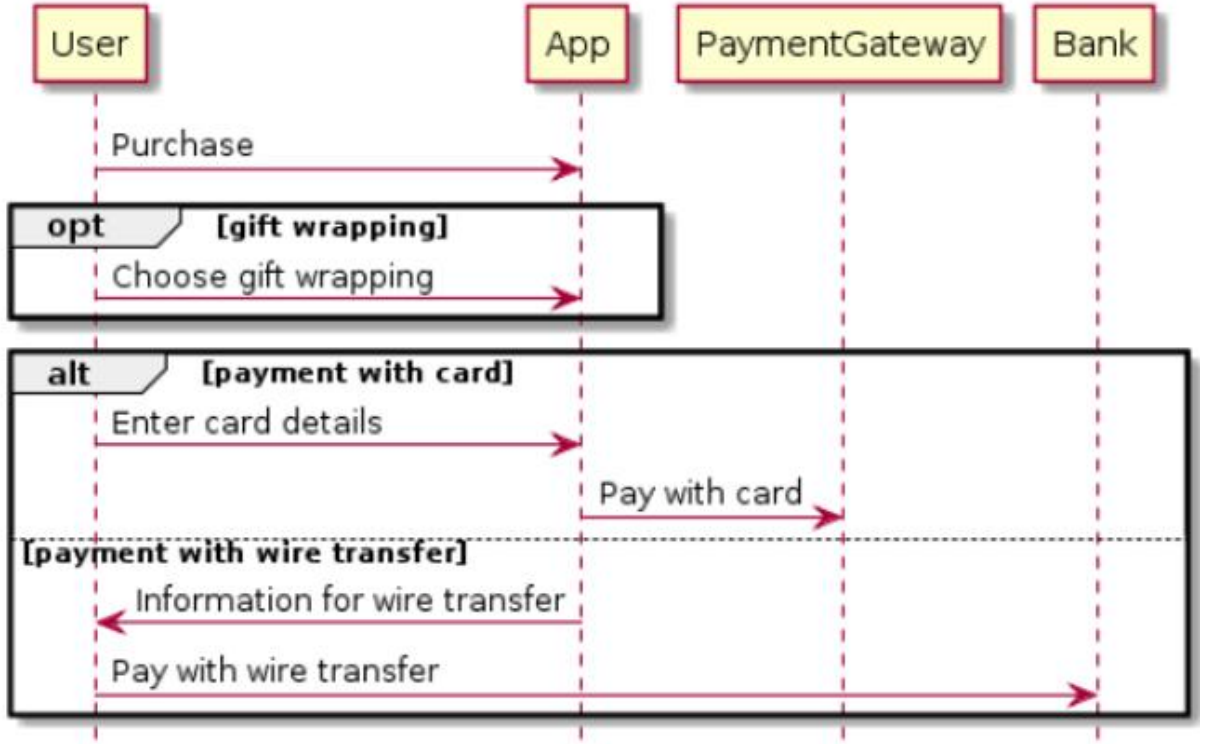


SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: *Combined Fragment*

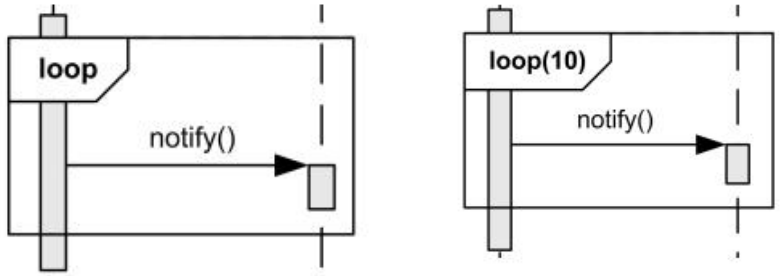
- ✓ Frequently used Combined fragments are:
  - alt – alternatives (if else)
  - opt – option (single condition)
  - loop - iteration
  - ref – reference

- ✓ The option combination fragment is used to indicate a sequence that will only occur under a certain condition, otherwise, the sequence won't occur.
- ✓ the option fragment is also represented with a rectangular frame where 'opt' is placed inside the name box.



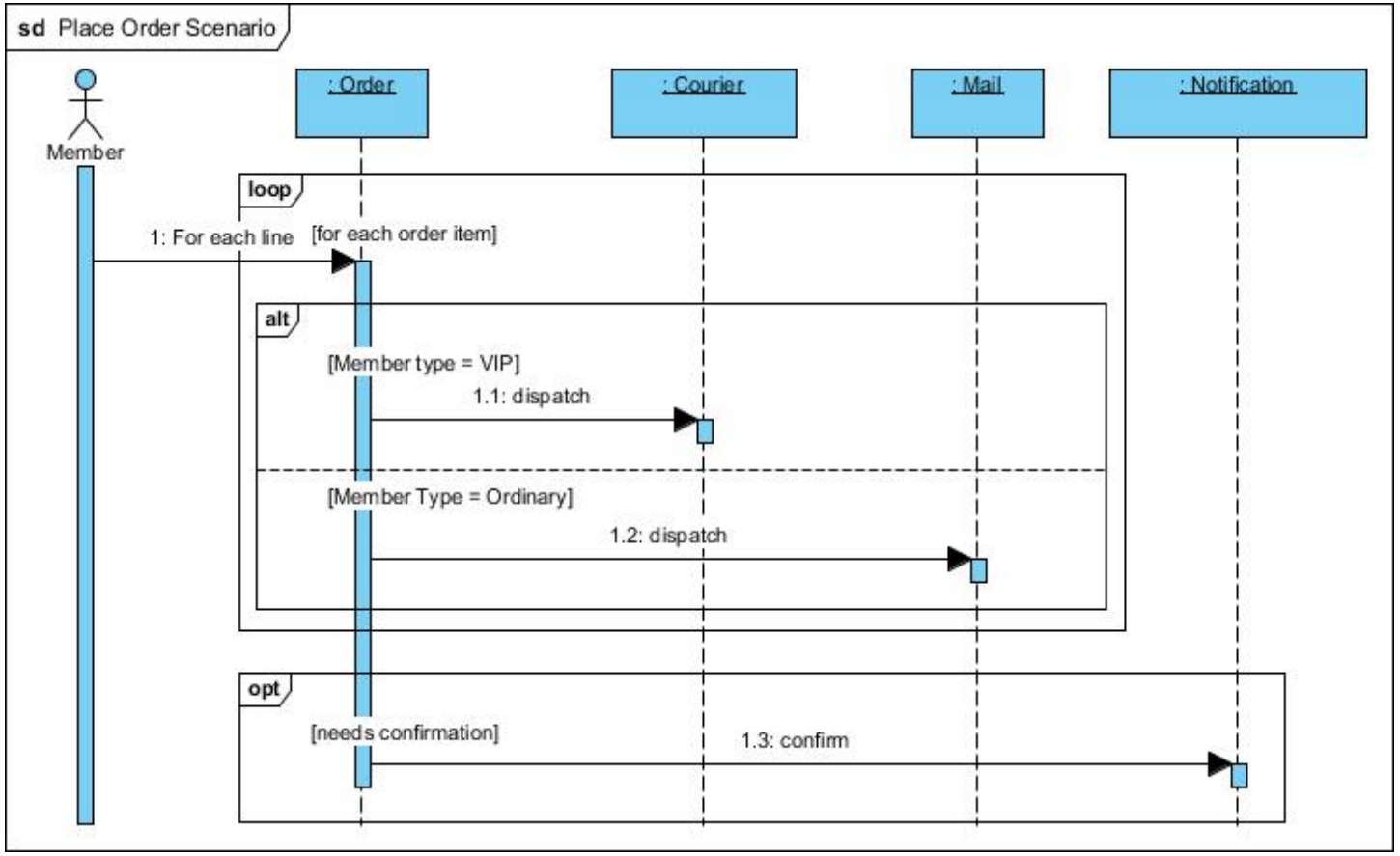
SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: *Combined Fragment*



- ✓ Frequently used Combined fragments are:
  - alt - alternatives
  - opt - option
  - **loop - iteration**
  - ref – reference

- ✓ loop means that the combined fragment represents a loop
- ✓ Loop could be controlled by either or both iteration bounds and a guard.
- ✓ If loop has no bounds specified, it means potentially infinite loop with zero as lower bound and infinite upper bound.



SEQUENCE DIAGRAMS (CONT'D)

Elements of a Sequence Diagram: *Combined Fragment*

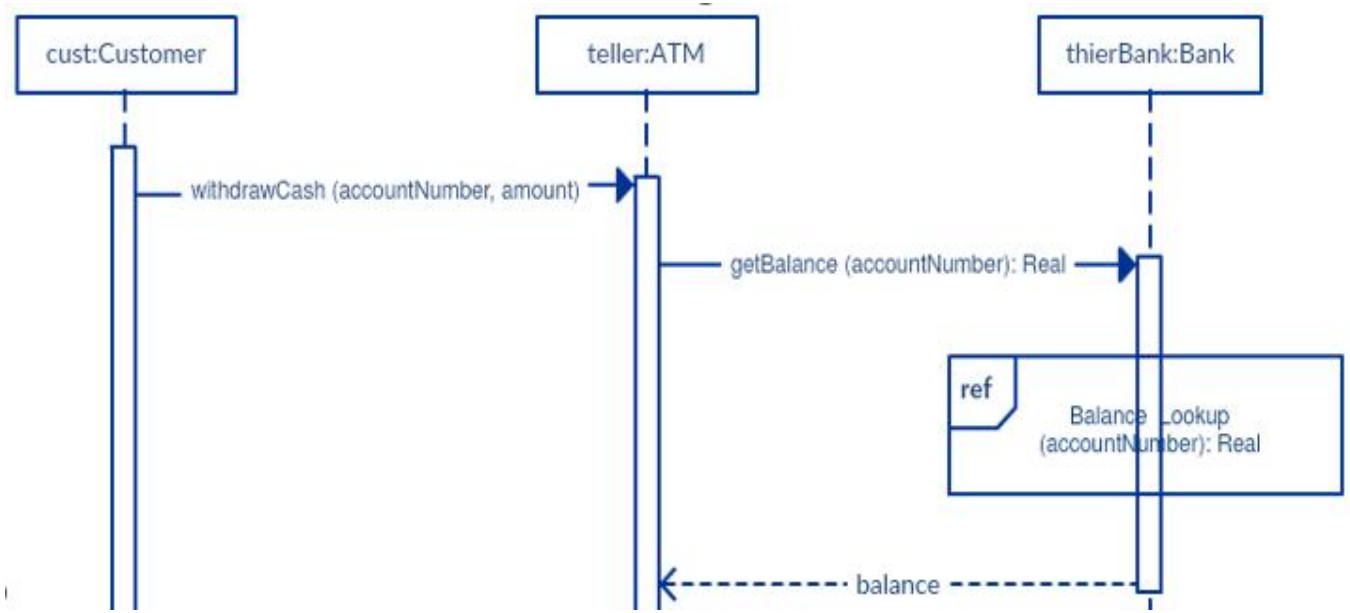
✓ Frequently used Combined fragments are:

- alt - alternatives
- opt - option
- loop - iteration
- **ref – reference**

✓ Provide a reference to another diagram.

✓ You can use the ref fragment to manage the size of large sequence diagrams.

✓ It allows you to reuse part of one sequence diagram in another



EXERCISE

Draw the sequence diagram for the purchase product use case.

- ✓ The customer selects a product to purchase.
- ✓ The system calculates the total price, including taxes and shipping costs.
- ✓ The customer chooses to proceed to checkout.
- ✓ The system prompts the customer for shipping and payment information.
- ✓ The customer provides shipping and payment details.
- ✓ The system verifies the payment information.
- ✓ The system updates the product's inventory.
- ✓ The system generates an order confirmation.
- ✓ The system displays the order confirmation to the customer.

ACTIVITY DIAGRAMS

- Fancy flowchart
  - A UML activity diagram helps to visualize a certain use case at a more detailed level.
  - It is a behavioral diagram that illustrates the flow of activities through a system.

Activity Diagram Symbols

Symbol	Name	Use
	Start/ Initial Node	Used to represent the starting point or the initial state of an activity
	Activity / Action State	Used to represent the activities of the process
	Action	Used to represent the executable sub-areas of an activity

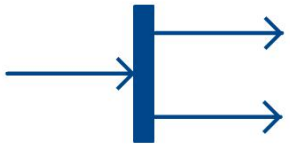
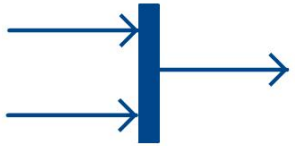
ACTIVITY DIAGRAMS

Activity Diagram Symbols

Symbol	Name	Use
	Control Flow	Used to represent the flow of control from one action to the other
	Object Flow / Control Edge	Used to represent the path of objects moving through the activity
	Activity Final Node	Used to mark the end of all control flows within the activity
	Flow Final Node	Used to mark the end of a single control flow
	*Decision Node*	Used to represent a conditional branch point with one input and multiple outputs

Activity Diagrams

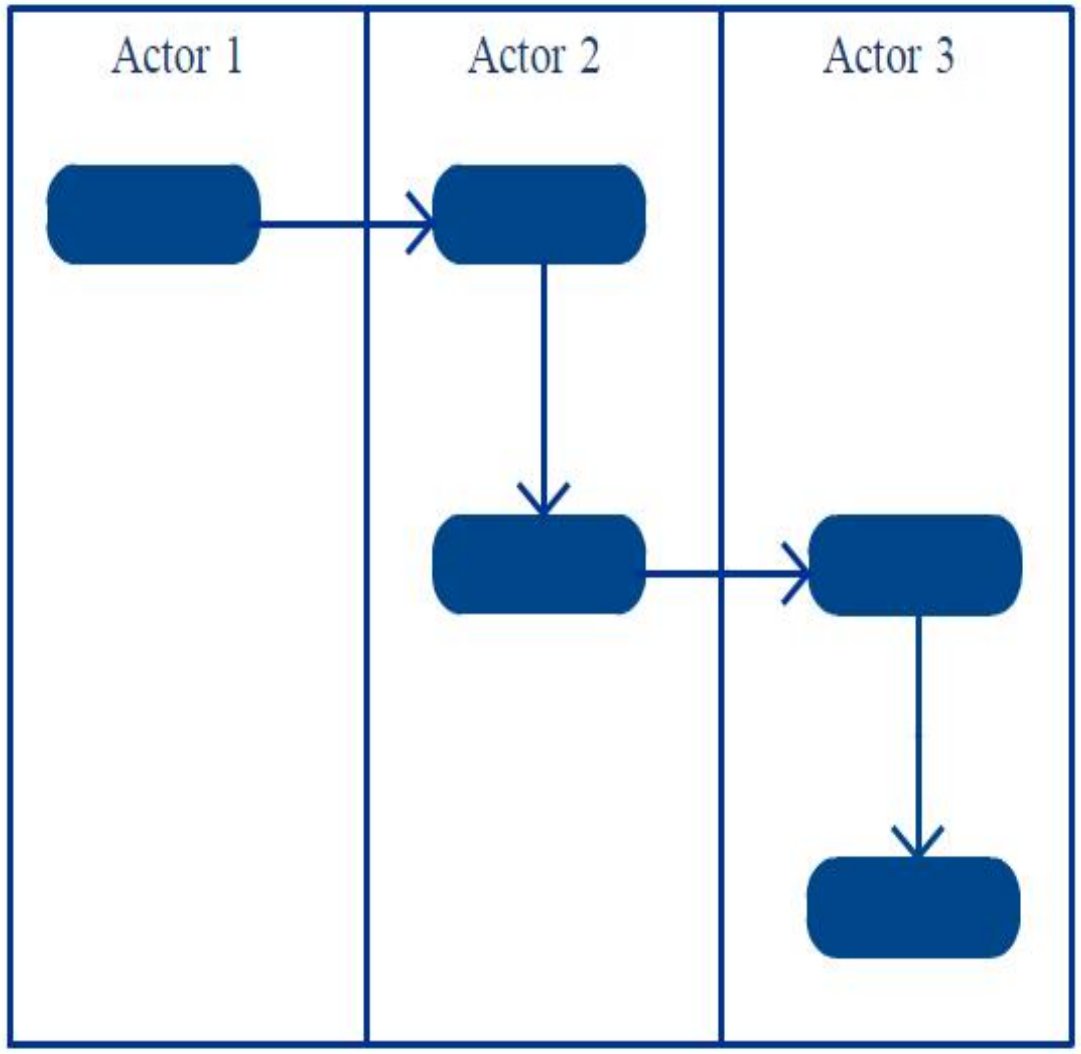
Activity Diagram Symbols

Symbol	Name	Use
	Fork	Used to represent a flow that may branch into two or more parallel flows
	Merge	Used to represent two inputs that merge into one output
	Signal Sending	Used to represent the action of sending a signal to an accepting activity
	Signal Receipt	Used to represent that the signal is received
	Note/comments	Used to add relevant comments to elements

ACTIVITY DIAGRAMS

Activity Diagrams with Swimlanes:

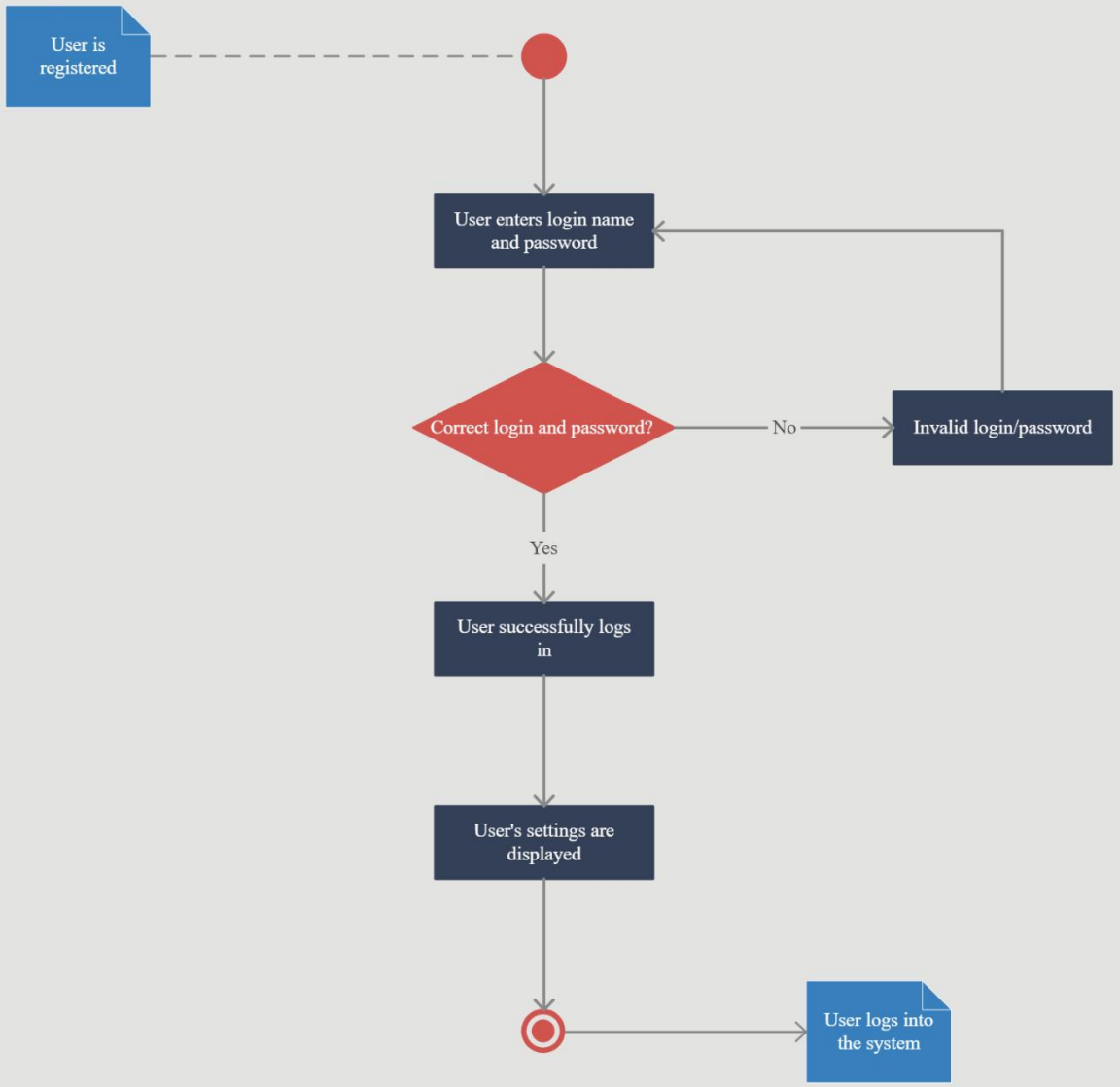
- ✓ also known as partitions – are used to represent or group actions carried out by different actors in a single thread.
- ✓ Tips to follow when using swimlanes:
  - Add swimlanes to linear processes. It makes it easy to read.
  - Don't add more than 5 swimlanes.
  - Arrange swimlanes in a logical manner.

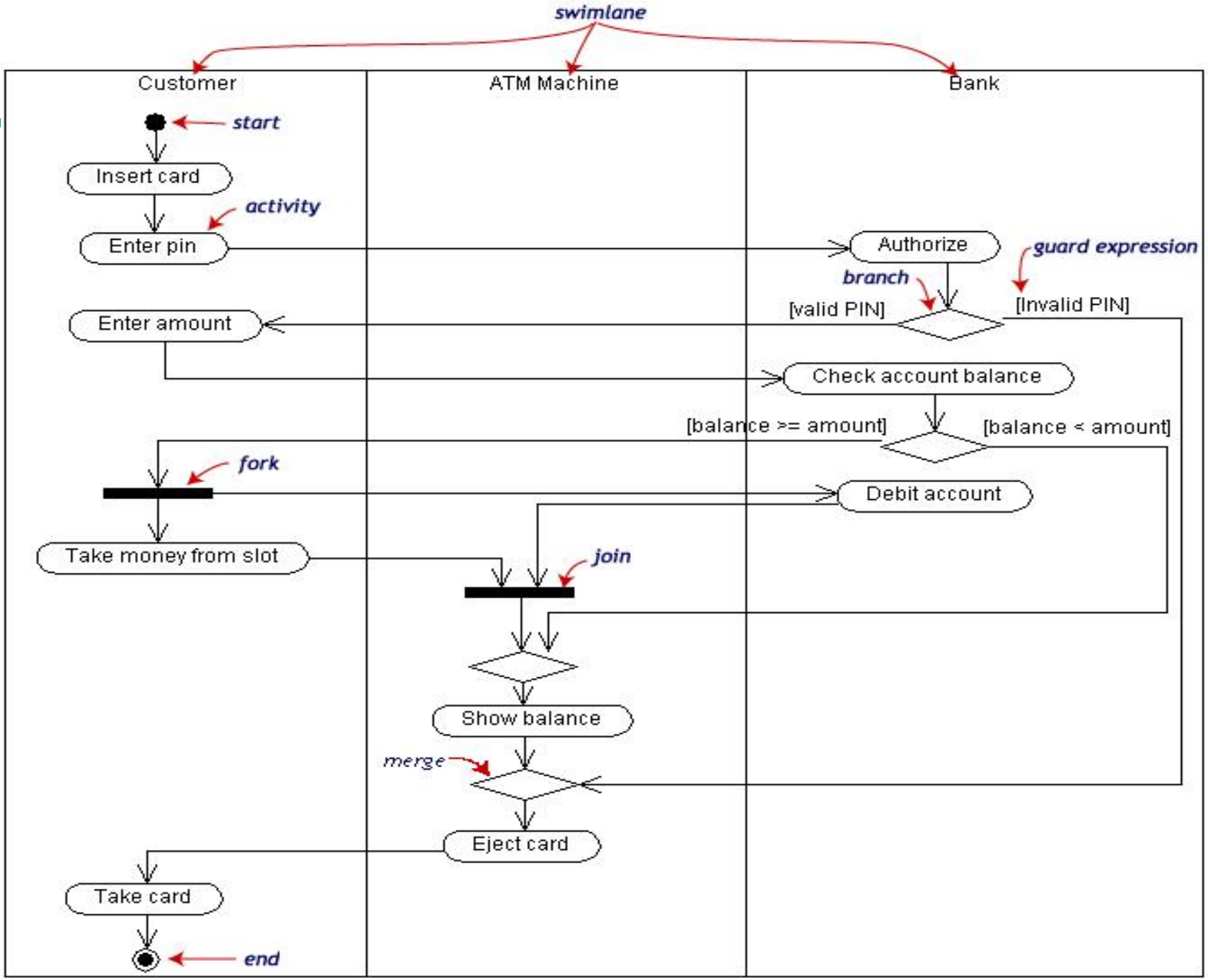


Activity Diagrams

How to Draw an Activity Diagram:

- ✓ **Step 1:** Figure out the action steps from the use case
- ✓ **Step 2:** Identify the actors who are involved
- ✓ **Step 3:** Find a flow among the activities
- ✓ **Step 4:** Add swimlanes




swimlane
Customer
ATM Machine
Bank
start
Insert card
activity
Enter pin
Authorize
guard expression
branch
[valid PIN]
[Invalid PIN]
Enter amount
Check account balance
[balance >= amount]
[balance < amount]
fork
Debit account
Take money from slot
join
Show balance
merge
Eject card
Take card
end

State Diagram

- ✓ State diagrams are used to describe the behavior of a system.
- ✓ State diagrams describe an object's possible states as events occur.
- ✓ It is important to note that having a State diagram for your system is not mandatory but must be defined only on a need basis (to understand the behavior of the object through the entire system)

STATE DIAGRAM

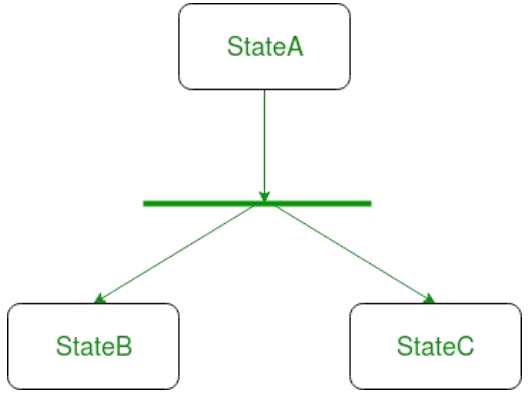
Components of State Diagram:

- ✓ **Initial state** – We use a black filled circle represent the initial state of a System or a class.

- ✓ **Transition** – We use a solid arrow to represent the transition or change of control from one state to another. The arrow is labelled with the event which causes the change in state.

- ✓ **State** – We use a rounded rectangle to represent a state. A state represents the conditions or circumstances of an object of a class at an instant of time.

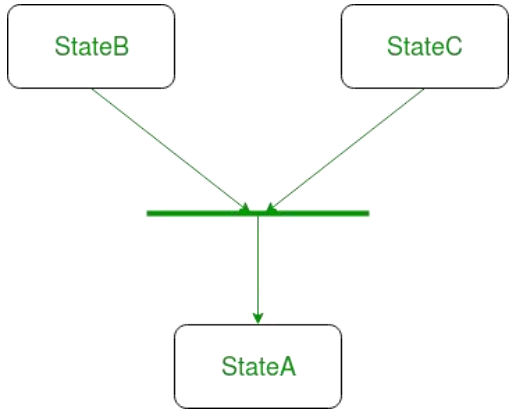
- ✓ **Fork** – We use a rounded solid rectangular bar to represent a Fork notation with incoming arrow from the parent state and outgoing arrows towards the newly created states. We use the fork notation to represent a state splitting into two or more concurrent states.



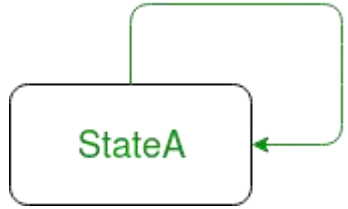
State Diagram

Components of State Diagram:

- ✓ **Join** – We use a rounded solid rectangular bar to represent a Join notation with incoming arrows from the joining states and outgoing arrow towards the common goal state. We use the join notation when two or more states concurrently converge into one on the occurrence of an event or events.



- ✓ **Self transition** – We use a solid arrow pointing back to the state itself to represent a self transition. There might be scenarios when the state of the object does not change upon the occurrence of an event. We use self transitions to represent such cases.

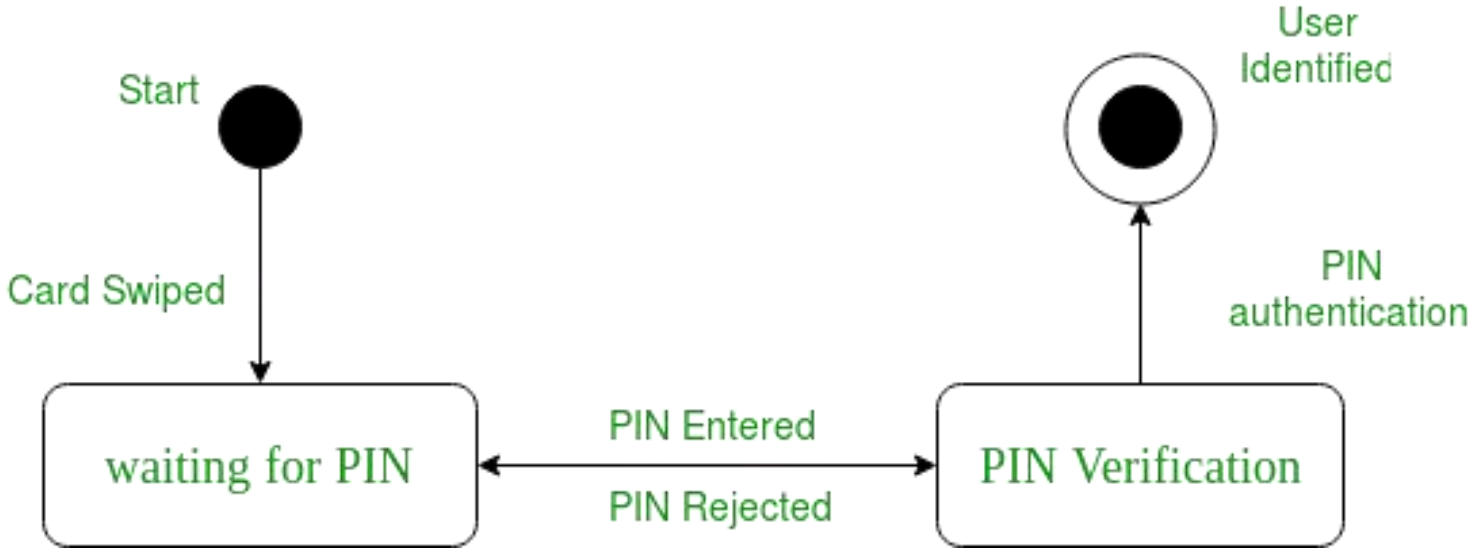


- ✓ **Final state** – We use a filled circle within a circle notation to represent the final state in a state machine diagram.

STATE DIAGRAM

Steps to draw a state diagram :

- ✓ Identify the initial state and the final terminating states.
- ✓ Identify the possible states in which the object can exist (boundary values corresponding to different attributes guide us in identifying different states).
- ✓ Label the events which trigger these transitions.



a state diagram for user verification on POS payment

COMPONENT DIAGRAM

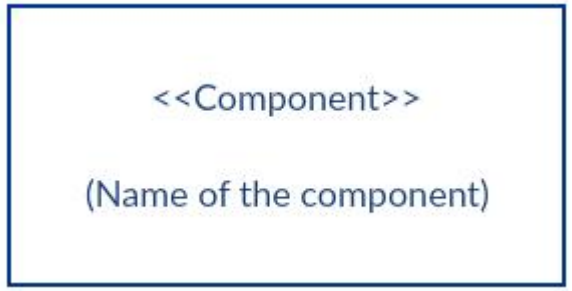
- A component diagram displays the structural relationship of components of a software system.
- used to visualize the organization of system components and the dependency relationships between them
- These are mostly used when working with complex systems with many components.
- Components communicate with each other using interfaces. The interfaces are linked using connectors.
- The components can be a software component such as a database or user interface; or a hardware component such as a circuit, microchip or device; or a business unit such as supplier, payroll or shipping.

COMPONENT DIAGRAM (CONT'D)

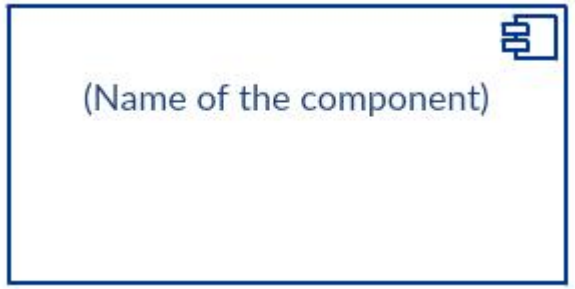
Component Diagram Symbols

✓ Components

- There are three ways the component symbol can be used.



Rectangle with the component stereotype (the text <<component>>



Rectangle with the component icon in the top right corner and the name of the component.

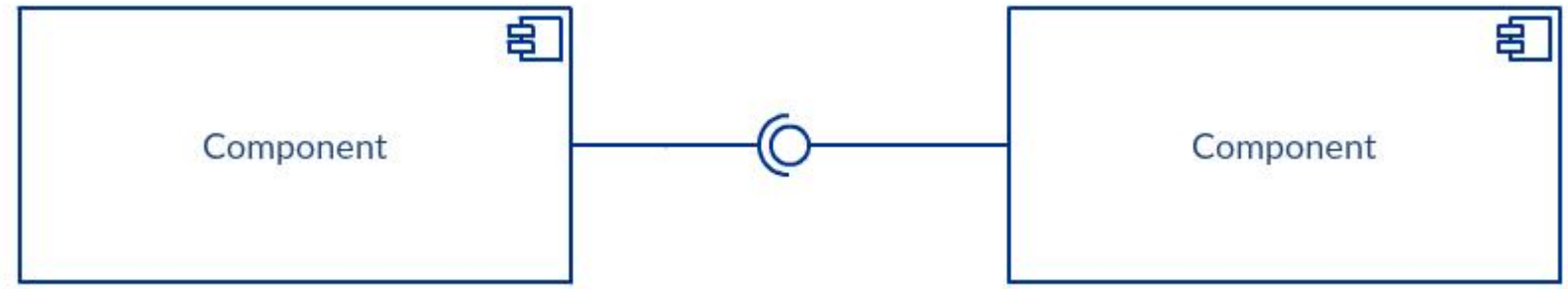
Rectangle with the component icon and the component stereotype.

COMPONENT DIAGRAM (CONT'D)

Component Diagram Symbols

✓ Provided Interface and the Required Interface

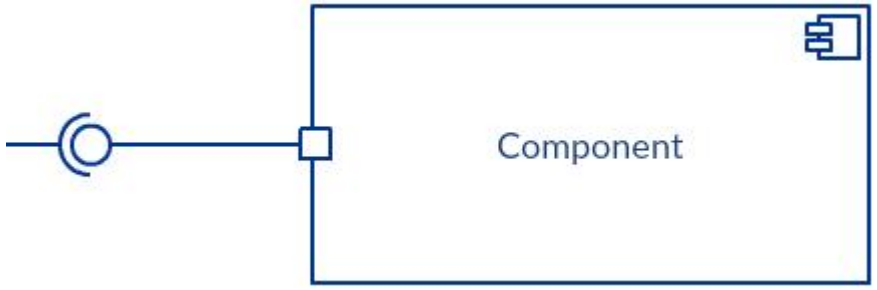
- Interfaces in component diagrams show how components are wired together and interact with each other.
- The assembly connector allows linking the component's **required interface** (represented with a semi-circle and a solid line) with the **provided interface** (represented with a circle and solid line) of another component.



COMPONENT DIAGRAM (CONT'D)

Component Diagram Symbols

- ✓ Port
  - Port (represented by the small square at the end of a required interface or provided interface) is used when a component needs to interact with another component or external system.



- ✓ Dependencies
  - Shown using dashed line

COMPONENT DIAGRAM (CONT'D)

How to Draw a Component Diagram

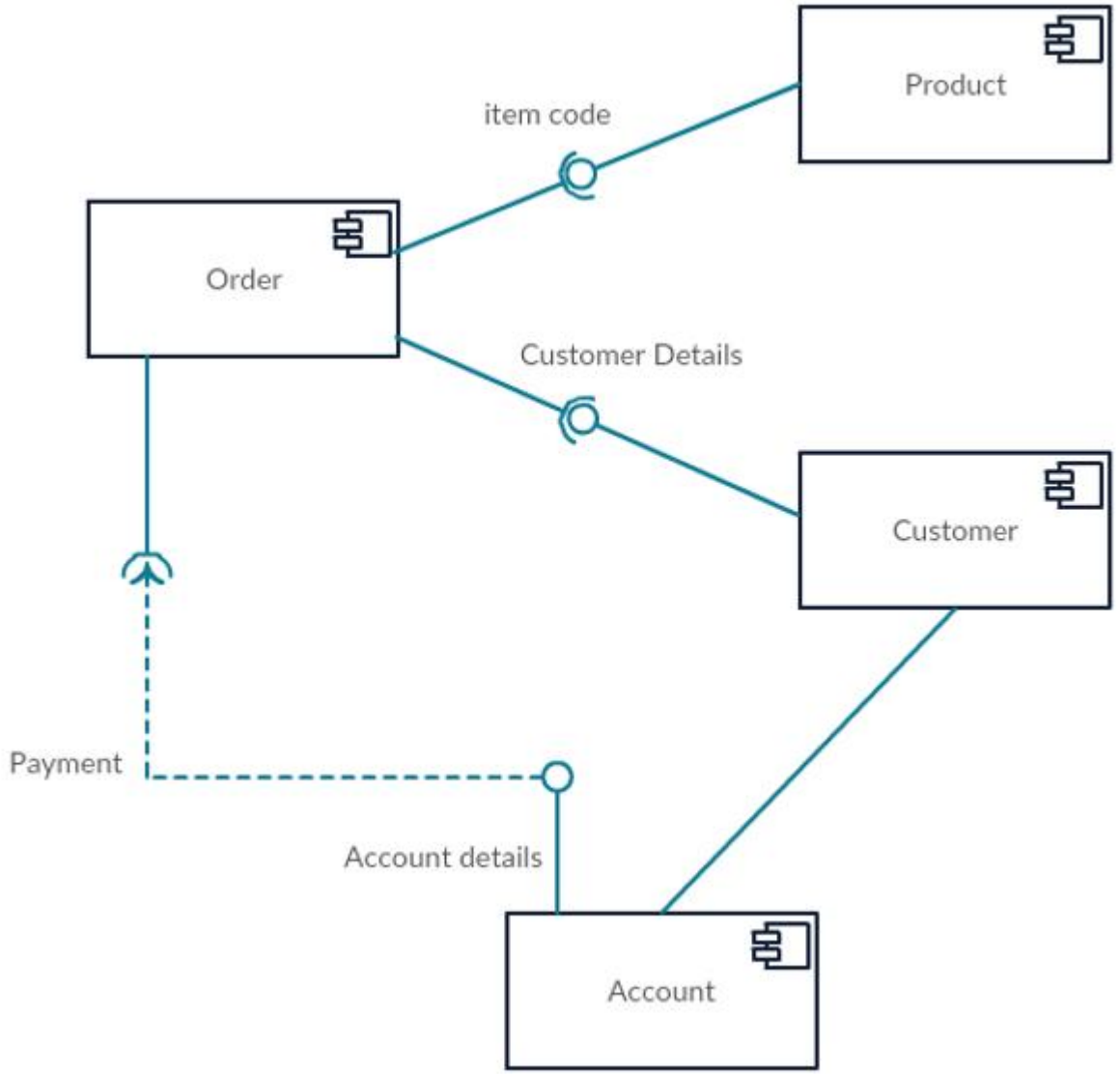
Step 1: figure out the purpose of the diagram and identify the artifacts such as the files, documents etc. in your system or application that you need to represent in your diagram.

Step 2: As you figure out the relationships between the elements you identified earlier, create a mental layout of your component diagram

Step 3: As you draw the diagram, add components first, grouping them within other components as you see fit

Step 4: Next step is to add other elements such as interfaces, classes, objects, dependencies etc. to your component diagram and complete it.

COMPONENT DIAGRAM (CONT'D)



On-line Shopping system

DEPLOYMENT DIAGRAM

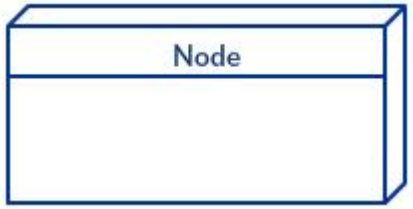
- ✓ Used to visualize the hardware processors/ nodes/ devices of a system, the links of communication between them and the placement of software files on that hardware.
- ✓ Using it you can understand how the system will be physically deployed on the hardware.

Deployment diagrams are useful when your software solution is deployed across multiple machines with each having a unique configuration.

DEPLOYMENT DIAGRAM

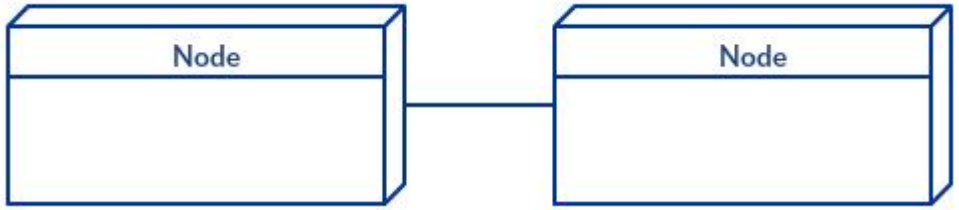
Deployment Diagram Notations

- ✓ Nodes
  - represented as a cube, is a physical entity that executes one or more components, subsystems or executables. A node could be a hardware or software element.



- ✓ Artifacts
  - represented as a cube, is a physical entity that executes one or more components, subsystems or executables.

- ✓ Communication Association
  - This is represented by a solid line between two nodes.

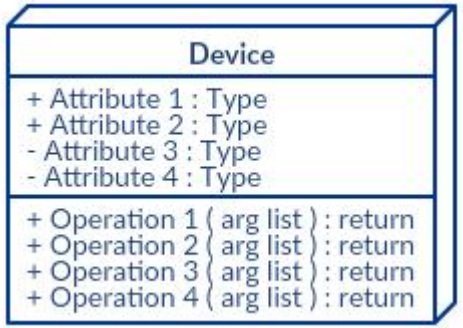


DEPLOYMENT DIAGRAM

Deployment Diagram Notations

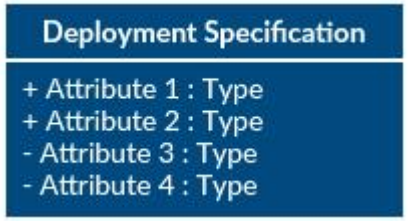
✓ Devices

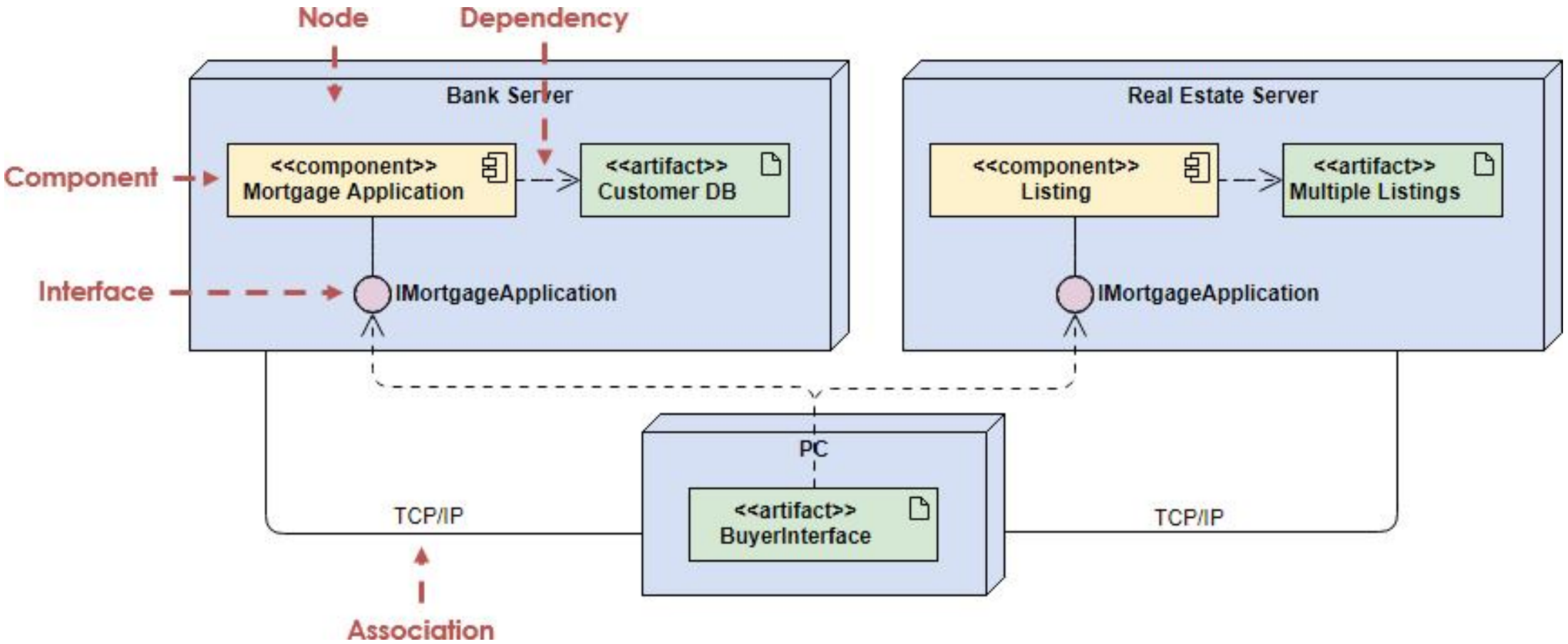
- A device is a node that is used to represent a physical computational resource in a system. An example of a device is an application server.



✓ Deployment Specifications

- Deployment specifications is a configuration file, such as a text file or an XML document. It describes how an artifact is deployed on a node..




Node
Dependency
Bank Server
Real Estate Server
Component
<<component>>
Mortgage Application
<<artifact>>
Customer DB
<<component>>
Listing
<<artifact>>
Multiple Listings
Interface
IMortgageApplication
IMortgageApplication
PC
<<artifact>>
BuyerInterface
TCP/IP
TCP/IP
Association